

MLOps Application

Github Repo

<https://github.com/deployml-core/deployml/tree/main>

Project Description

Direct from the paper is this: “One potential barrier academic departments might face in offering MLOps is the lack of instructors with hands-on industry experience. Hiring adjuncts who work in the field can be difficult, particularly in high cost-of-living areas. To make it simpler for prospective instructors to teach a course in MLOps, we hope to create an easy-to-use, scalable, modular, and lightweight library for seamlessly deploying an infrastructure in the cloud, integrated with learning modules that can be used for assignments and exercises in the classroom. An application for quickly deploying a basic low-cost MLOps infrastructure in the cloud, combined with exercises to promote computational learning but reduce friction in the debugging process, may remove some barriers for academic departments that lack the necessary resources for offering a class on this topic, but that understand the value of this topic to their students. This is an area of future work and is currently in development.”

We would like an application that makes it easy for students and instructors to deploy a set of commonly used MLOps tools in GCP, AWS, and Azure, to hopefully avoid unnecessary pain related to OS/dependency issues. There will be plenty of time for students to deal with those struggles on-the-job. For the classroom, however, the focus should be on getting those tools to work in the cloud as quickly as possible in order to start building.

Project Goals

This project has two main goals (or deliverables, if you want to use corporate lingo, which I prefer not to use):

- A software application with documentation and website
- Paper for publication

The application will be completed in phases, described in the next section, and each phase may have one or more releases. The general timeline is to complete the first phase by early September 2025, and phase 2 by December 2025. Phase 3, which is for Azure, is less of a priority given the lack of popularity of Azure.

Publication will be either through JOSS, pyopensci, or a journal.

Project Phases

Phase 1

The goal of the first phase of the project is to get at least one tool for each stage of the MLOps pipeline working in GCP, test the application, complete documentation for this phase, and create a website which includes an end-to-end example.

Phase 2

The goal of the second phase is to extend to AWS, conduct testing, modify documentation, website, and example.

Phase 3

The goal of the third phase is to extend to Azure, modify documentation, website, and example.

MLOps Tools & App Features

Tools

Here is a list of the tools we would like to deploy for each MLOps stage. We can add or subtract from this list, so long as we have justification for it. To add a tool, we should confirm that it is “popular”.

APP FEATURES	TOOLS	WHAT DOES THIS MEAN?
Experiment Tracking	MLFlow	Tracking server is running in cloud along with SQL instance and database
Artifact Tracking	MLFlow	Storage bucket is provisioned and tracking server is deployed
Model Registry	MLFlow	Storage bucket is provisioned and tracking server is deployed
Model Serving	Online: Fastapi (Optional) Batch: use an	Fastapi: a container with fastapi and model scoring code to run

	Orchestration tool from below (Optional) Online AI: Triton	with cloud run or on GKE (Optional) Batch: a container with model scoring code to run on a schedule using an orchestration tool
Model Monitoring	From scratch using tables, scripts, and a Grafana dashboard.	Bigquery or Postgres tables for storing metrics and a grafana dashboard for visualizing those metrics. A container running on a schedule to create the metrics. Used from scratch python code for drift metrics
Operational Monitoring	Grafana	A container that runs Grafana in cloud run and connects to monitoring metrics tables
Feature Store	From scratch	BigQuery tables for storing features
Data Versioning	Not included	
Fairness		
Explainability		
Security		
Orchestration		
Cost Estimation		

App Structure

GCP

Provisioning the infrastructure requires these steps:

1. Enabling APIs in GCP
2. Building docker images in GCP
3. Creating services in GCP

- a. Postgres database instance w/tables
- b. Bigquery tables
- c. Object storage for artifacts, including docker images
- d. Cloud run services for MLFlow, FastAPI (model serving), Grafana, using the docker images

The app has the following functionality:

- System checks for checking dependencies on local machine (run_doctor and check_system and doctor command)
- Project initialization which enables APIs, creates an empty configuration file and example Dockerfiles that can be used (init command)
- Docker image building that builds all images in the projects docker/ folder, in GCP (build_images)
- Deployment of infrastructure that uses a specified configuration.yaml file and creates the Terraform templates and applies the infrastructure in GCP (deploy). The configuration file specifies which parts of the infrastructure to provision.
- Infrastructure metadata extraction to get all connection information such as URLs (get_urls)
- Teardown which is just a wrapper around Terraform destroy (destroy)

The structure of the repo is below. While there are more folders than what I've listed here, I think they can safely be ignored, and eventually many of them can probably be deleted.

FOLDER	SUBFOLDER	PURPOSE
docs/		Documentation files for the website
example/	config/	Example configuration files to test the application. Focus on gcp-sample.yaml or gcp-sample-slim.yaml.
notebooks/		Notebooks that can be used to test the application. Focus on notebook_cli_demo.ipynb
src/deployml/	cli/	Code for the CLI commands
src/deployml/	diagnostics/	Code for the doctor command
src/deployml/	docker/	Example dockerfiles that get shipped with the application
src/deployml/	notebook/	Code for Jupyter notebook functions. These are for running some of the deployml commands interactively in a notebook rather than through

		the command line.
src/deployml/	templates/	Terraform templates. These are written with jinja. They are used to dynamically create the Terraform template files that are used for deploying the infrastructure. Focus on /templates/gcp/cloud_run
src/deployml/	terraform/modules/	These are the terraform modules that are actually used for filling in the templates. Focus on bigquery/ cloud_sql_postgres/ fastapi/ mlflow/ grafana/ teardown/
src/deployml/	utils/	These are all of the helper functions that are needed for the CLI or notebook functions.

Problems

While working through the application I have had an issue with getting cloud run to work correctly. The terraform templates will live in the .deployml/config_name folder, and the lock file might sometimes need to be deleted if trying to deploy the infra multiple times.

You will notice many other files and templates for things like feast, cron, weights & biases. There are also VM and Local options that are built-in. Leave them for now, but I may decide to remove these later to streamline the application and just have it focus on GCP using Cloud Run, and later AWS.

What Needs to be Completed

Tip: use "pip install -e ." to install the library while developing

What I was trying to finish up is the following:

1. Get the grafana image to work in Cloud Run.

2. Get the mlflow image to work in Cloud Run (it was working, but then suddenly it was not, and I'm not sure why).
3. Auto-creation of Bigquery tables with specific schemas, which are already defined in the repo, for:
 - a. Features
 - b. Predictions
 - c. Ground truth
 - d. Drift metrics
4. .env file generation which will have all of the infrastructure URLs
5. End-to-end workflow example.
 - a. Example training set to put into Bigquery
 - b. Script for training model, with MLFlow for tracking experiments
 - c. Model registration
 - d. Script for creating features, store in offline_features table in Bigquery
 - e. Script for model serving, which is used by the FastAPI container. Pulls latest registered model from MLFlow.
 - f. Script for making predictions with deployed model in FastAPI, and stores predictions in predictions table in Bigquery.
 - g. Script for storing "fake" ground_truth data in ground_truth table in Bigquery
 - h. Script for creating drift metrics and storing in drift_metrics table in Bigquery
 - i. Use Grafana to make a model monitoring dashboard

Documentation

Use MKdocs to populate the docs and MKdocstrings to populate the API documentation (https://docs.pydantic.dev/latest/api/base_model/, <http://mkdocs.org/>)

Pipeline

